**Практическая работа №7 (часть 2)**

**SQLAlchemy. Создание класса (таблицы)**

Для того, чтобы рассмотреть ORM (object relation mapping) представление данных изначально можно рассмотреть базовые возможности библиотеки SQLAlchemy – объекты классов declarative_base() и session().

Прежде чем перейти к созданию объектов необходимо настроить подключение к серверу. В SQLAlchemy предложено пользоваться ссылкой в виде URL.

```
import sqlalchemy as db
import mysql.connector
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker
from sqlalchemy.orm import declarative_base
engine = db.create_engine("mysql+mysqlconnector://user_name:password@server_name/database_name")
con=engine.connect()
```

Рисунок 1 – Создание соединения с сервером

Класс declarative_base позволяет определить класс, который в дальнейшем будет отображаться, как таблица на стороне базы данных. В текущей базе данных создана таблица Loyalty, которая будет содержать информацию о картах лояльности авиакомпаний. Для того, чтобы сопоставить класс с таблицей использовался атрибут класса _tablename_, далее были перечислены будущие столбцы, которым присваивается значение column().

Объект сессии session() нужен для того, чтобы работать с базой данных bind: привязывает сессию базе данных к определенному движку, который применяется для установки подключения. Результатом функции sessionmaker() является класс Session. После этого можно создать объект этого класса и через него взаимодействовать с базой данных.

```
Base = declarative_base()
class Loyalty(Base):
    __tablename__ = 'loyalty'
    id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    id_air =Column(Integer, foreign_key=True)

Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()
```

Рисунок 2 – Создание класса (таблицы)

Далее созданы объекты моделей, содержащие строки в базе данных: bronze, silver, gold , которые передаются в метод add() объекта Session. После добавления для подтверждения изменений у объекта Session вызывается метод commit().

Чтобы получить данные из класса (таблицы) можно воспользоваться методом query() для класса session, позволяющим вывести данные обо всех записях, т.к. в конце добавлено .all(). В цикле выводятся соответствующие записи.

После этого сессия закрывается.

```
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()
bronze = Loyalty(name='Bronze', id_air=3)
silver = Loyalty(name='silver', id_air=3)
gold = Loyalty(name='gold', id_air=1)
session.add(bronze)
session.add(gold)
session.add(silver)
session.commit()
loyalty = session.query(Loyalty).all()
for loyalty in loyalty:
    print(f"ID: {loyalty.id}, Name: {loyalty.name}, id_air: {loyalty.id_air}")
session.close()
```

Рисунок 3 – Добавление значений в таблицу, вывод значений на экран

Для того, чтобы обновить данные необходимо сначала занести в объект new результат отработки запроса на выборку. Затем атрибуту объекта (столбцу) присвоить нужное значение. Благодаря оператору commit операция успешно выполняется. Далее с помощью print() производится проверка изменений в базе данных.

```
new = session.query(Loyalty).filter(Loyalty.id==16).first()
new.id_air = 4
session.commit()

query_new = session.query(Loyalty).filter(Loyalty.id==16).first()
print(f"{query_new.id},{query_new.name}, {query_new.id_air}")
```

Рисунок 4 – Обновление данных с помощью класса session

Удаление возможно с помощью метода delete(), который соответствует атрибуту session. Атрибут delete позволяет удалить запись, хранящуюся в объекте, представленном запросом. Далее производится проверка и соответствующие записи не выводятся.

```
query_to_delete = session.query(Loyalty).filter(Loyalty.id==15).first()
session.delete(query_to_delete)
session.commit()

query_two=session.query(Loyalty).all()
for query_two in query_two:
  print(f"{query_two.id}, {query_two.name}, {query_two.id_air}")
session.close()
```

Рисунок 5 – Удаление записей с помощью класса session

***Задание.***

Создать в своей базе данных 2 таблицы, добавить в них информацию и вывести на экран все изменения. Обновить и удалить данные из таблиц, вывести изменения на экран.